AgentSZZ: Teaching the LLM Agent to Play Detective with Bug-Inducing Commits

Yunbo Lyu[1], Jieke Shi[1], Hong Jin Kang[2], Ratnadira Widyasari[1], Junda He[1], Yuqing Niu[1], Chengran Yang[1*], Junkai Chen[1], Zhou Yang[3], Julia Lawall[4], David Lo[1]
[1]Singapore Management University, Singapore [2]University of Sydney, Sydney, Australia [*]Corresponding author
[3]University of Alberta, Edmonton, Canada [4]Inria-Paris, Paris, France
{yunbolyu,jiekeshi,ratnadiraw,yuqingniu,cryang,junkaichen,davidlo}@smu.edu.sg
hongjin.kang@sydney.edu.au,jundahe.2022@phdcs.smu.edu.sg,zy25@ualberta.ca,julia.lawall@inria.fr

arXiv:2604.02665v1 [cs.SE] 3 Apr 2026

Abstract

The SZZ algorithm is the dominant technique for identifying bug-inducing commits and underpins many software engineering tasks, such as defect prediction and vulnerability analysis. Despite numerous variants, including recent LLM-based approaches, performance remains limited on developer-annotated datasets (e.g., recall of 0.552 on the Linux kernel). A key limitation is the reliance on *git blame*, which traces line-level changes within the same file, failing in common scenarios such as ghost and cross-file cases—making nearly one-quarter of bug-inducing commits inherently untraceable. Moreover, current approaches follow fixed pipelines that restrict iterative reasoning and exploration, unlike developers who investigate bugs through an interactive, multi-tool process.

To address these challenges, we propose AgentSZZ, an agent-based framework that leverages LLM-driven agents to explore repositories and identify bug-inducing commits. Unlike prior methods, AgentSZZ integrates task-specific tools, domain knowledge, and a ReAct-style loop to enable adaptive and causal tracing of bugs. A structured compression module further improves efficiency by reducing redundant context while preserving key evidence. Extensive experiments on three widely used datasets show that AgentSZZ consistently outperforms state-of-the-art SZZ algorithms across all settings, achieving F1-score gains of up to 27.2% over prior LLM-based approaches. The improvements are especially pronounced in challenging scenarios such as cross-file and ghost commits, with recall gains of up to 300% and 60%, respectively. Ablation studies show that task-specific tools and domain knowledge are critical, while compression tool outputs reduces token consumption by over 30% with negligible impact. Replication package is available.

CCS Concepts

• **Software and its engineering → Maintaining software**.

Keywords

SZZ algorithm, agent, large language model

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Preprint, Arxiv
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

ACM Reference Format:
Yunbo Lyu[1], Jieke Shi[1], Hong Jin Kang[2], Ratnadira Widyasari[1], Junda He[1], Yuqing Niu[1],, Chengran Yang[1*], Junkai Chen[1], Zhou Yang[3], Julia Lawall[4], David Lo[1]. 2026. AgentSZZ: Teaching the LLM Agent to Play Detective with Bug-Inducing Commits. In . ACM, New York, NY, USA, 12 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

SZZ algorithms have been proposed since 2005 [33] and have become a foundational technique with broad impact in the software engineering (SE) community [1]. Given a bug-fixing commit, SZZ identifies the bug-inducing commits that introduced the defect. They have been widely used in many downstream tasks such as defect prediction [13, 15, 28, 34, 46], analysis of how bugs are introduced [5, 6, 39], and vulnerability version identification [4, 8]. More recently, security code generation benchmarks also rely on SZZ algorithms to identify vulnerability-inducing commits [7].

Many variants of SZZ algorithms have been proposed to improve the accuracy of bug-inducing commit identification. AG-SZZ [18] filters cosmetic changes such as blank lines and comments; MA-SZZ [10] excludes meta, branch, and property changes; and RA-SZZ [24] detects and filters refactoring changes. More recently, LLM4SZZ [36] leverages LLMs to assess blame candidates with expanded context, achieving state-of-the-art (SOTA) results across multiple datasets. Despite these advances, performance remains far from satisfactory on developer-annotated datasets [23, 30, 42]. For example, LLM4SZZ achieves a recall of only 0.552 on the Linux kernel dataset, indicating that nearly half of the bug-inducing commits remain unidentified.

The **fundamental limitation** is that existing SZZ algorithms rely heavily on *git blame* to trace modified lines in bug-fixing commits. As a result, they fail to identify bug-fixing commits without deleted or modified lines (i.e., ghost commits [29]) and bug-inducing commits located in different files (i.e., cross-file cases [23]). Prior studies show that, in the Linux kernel dataset, 17.47% of bug-fixing commits are ghost commits and 7.20% are cross-file cases [23], meaning nearly one-quarter of bug-inducing commits are inherently untraceable by blame-based methods. However, in practice, developers are not limited to a single tool, but **use diverse tools** (e.g., *git log*) to gather context and understand bugs [19, 27]. Current SOTA LLM4SZZ [36] still follow fixed pipelines that predefine what to inspect, limiting iterative reasoning and exploration. In contrast, developers follow an **information foraging process** [20], exploring promising paths and backtracking when encountering uninformative changes.

Agent-based approaches are a natural fit for this problem, as they can simulate developers' dynamic and interactive investigation processes, enabling models to actively explore repositories and follow information scents [47, 49, 50]. However, **they are not a silver bullet**. Our exploration with mini-SWE-Agent [47], a powerful SWE agent with flexible bash interaction, shows poor performance. Several challenges remain: ❶ agents struggle with tool selection, ❷ actions lack domain-specific guidance, and ❸ tool outputs can be large, leading to excessive token consumption, increased latency, and noisy context that may hinder reasoning.

To address these challenges, we propose AgentSZZ, as shown in Figure 1, an agent-based framework that uses LLM-driven agents to interact with repositories and identify bug-inducing commits akin to a detective. Unlike prior approaches that rely primarily on *git blame*, AgentSZZ interacts with repositories through five task-specific tools designed for bug-inducing commit identification. Given a fix commit, it iteratively explores the repository via a ReAct-style loop [48], using GPT-5-mini as the backbone model with a maximum of 15 interaction turns, leveraging these tools and domain knowledge to trace causal evidence. A structured compression module further improves efficiency by reducing redundant and noisy context while preserving key information.

The evaluation results on three widely used datasets show that AgentSZZ consistently outperforms SOTA SZZ algorithms across all settings. AgentSZZ achieves substantial gains, improving F1-score by up to 27.2% over prior SOTA LLM4SZZ, and maintains strong performance even under the same backbone model, indicating that the improvements stem from the agent design rather than the choice of LLM. These gains are especially pronounced in challenging scenarios such as cross-file and ghost commits, where recall improves by up to 300% and 60%, respectively, highlighting the ability of AgentSZZ to improve the fundamental limitations of blame-based methods. Ablation studies further reveal that task-specific tools and domain knowledge are central to performance, while the compression module significantly improves efficiency by reducing token consumption by over 30% with negligible impact. AgentSZZ also generalizes well across datasets and programming languages without dataset-specific tuning.

In summary, we make the following contributions: (1) We propose AgentSZZ, an agent-based framework that leverages LLM-based agents to interact with repositories and identify bug-inducing commits, enabling **adaptive and causal tracing** beyond the limitations of existing SZZ algorithms. (2) We design five task-specific tools, encode domain knowledge, and introduce a structured compression module to improve the agent's reasoning capability and efficiency in repository exploration. (3) We conduct extensive experiments on three widely used datasets, showing that AgentSZZ significantly outperforms SOTA SZZ algorithms, particularly in challenging scenarios such as ghost commits and cross-file cases. (4) We provide a replication package including code and data.

The remainder of this paper is organized as follows. Section 2 provides background on SZZ algorithms and LLM-based agents for software engineering. Section 3 presents the design of AgentSZZ, Section 5 describes the experimental setup, and Section 5 reports the evaluation results. Section 6 offers further analysis and discussion, and Section 7 concludes the paper.

2 Background and Related Work

2.1 SZZ Algorithms

The SZZ algorithm [33] is the foundational approach for automated BIC identification. Given a bug-fixing commit, it traces deleted or modified lines back to the commits that last touched them, treating those commits as BIC candidates. The original implementation operates on CVS repositories using the annotate command; modern implementations adapt this principle to Git via *git blame*. While conceptually simple, the algorithm suffers from false positives, as not all changes to the fixed lines are semantically meaningful.

A series of refinements have been proposed to address these limitations. AG-SZZ [18] filters cosmetic changes such as blank lines and comments, and introduces annotation graphs to track line origins more precisely across revisions. MA-SZZ [10] extends this by excluding meta-changes (e.g., branch merges and property changes) that do not alter program behavior. RA-SZZ [24] further detects and filters refactoring operations using RefDiff [32] and Refactoring Miner [38], though this capability is limited to Java. When multiple candidates remain, R-SZZ and L-SZZ [12] apply simple heuristics, selecting the most recent commit or the one with the most changed lines, respectively.

More recent work shifts from heuristic filtering to learned or model-based ranking. Neural-SZZ [35] embeds changed lines using CodeBERT [14] and ranks candidates via a heterogeneous graph attention network, capturing semantic relationships beyond rule-based methods. LLM4SZZ [36] further leverages LLMs to assess blame candidates with expanded context, achieving state-of-the-art results across multiple datasets. However, it still operates as a static pipeline without iterative investigation.

2.2 LLM-Based Agents for Software Engineering

The application of LLMs in SE has evolved from static, single-turn code generation to dynamic, autonomous agents [16, 21]. Unlike traditional LLM applications that rely solely on internal model weights, LLM-based agents are augmented with reasoning frameworks (e.g., ReAct [48]), memory mechanisms, and the ability to interact with external environments via tools [43]. This evolution enables agents to tackle complex SE tasks across the Software Development Life Cycle, shifting from snippet-level synthesis to repository-level reasoning and maintenance [21].

Recent work on agents designed to navigate large codebases and resolve real-world issues, as exemplified by SWE-bench [17]. Systems such as Devin [9] and OpenHands [41] demonstrate end-to-end patch generation, while SWE-agent [47] introduces an Agent-Computer Interface (ACI) to bridge LLMs and terminal environments by structuring tool interactions. Lightweight variants such as mini-SWE-agent (a lightweight variant of SWE-agent) [47] demonstrate that minimal bash-based agents can achieve competitive performance. Other approaches enhance repository interaction through structured search or execution mechanisms, including AST-based navigation (AutoCodeRover [50]) and executable action spaces (CodeAct [40]). In contrast, pipeline-based methods such as Agentless [44] simplify execution by replacing autonomous planning with a fixed pipeline. Despite these advances, existing agents either rely on general-purpose exploration (e.g., bash-based interaction) or predefined pipelines, lacking task-specific tool design

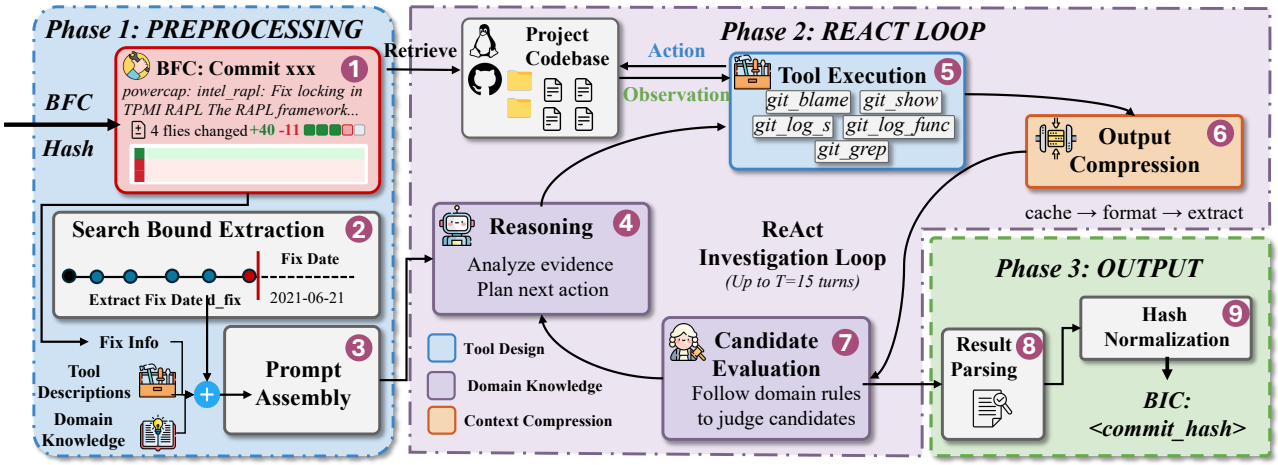

Figure 1: Overview of AgentSZZ, illustrating the end-to-end workflow from preprocessing (1-3), through the ReAct investigation loop (4-7), to output generation (8-9). Given a bug-fixing commit (BFC), the agent investigates the repository through tool interactions guided by domain knowledge, with context compression, and outputs the identified bug-inducing commit (BIC).

and domain knowledge for precise bug-inducing commit identification. AgentSZZ addresses this gap by combining structured, task-specific tools with domain knowledge to enable accurate bug-inducing commit identification.

3 AgentSZZ Approach

To address the limitations of existing SZZ algorithms and the challenges of applying agent-based approaches to bug-inducing commit identification, we propose AgentSZZ, an agent-based framework that leverages LLM-based agents to interact with repositories and identify bug-inducing commits, as illustrated in Figure 1. AgentSZZ performs **adaptive investigation** that more closely resembles how developers analyze bugs, in contrast to prior SZZ approaches that follow fixed pipelines. Inspired by the ReAct paradigm [48], the agent dynamically selects investigation strategies based on intermediate findings, enabling flexible repository exploration. Instead of relying on syntactic line matching, it **reasons about causal relationships** between commits and the bug, distinguishing superficial changes (e.g., refactorings) from those that introduce faulty logic. This enables AgentSZZ to handle challenging scenarios such as cross-file cases and refactoring-obscured bugs.

3.1 Design Process

The components of AgentSZZ were derived through an iterative, case-driven design process rather than predefined. To guide this process, we randomly sampled 50 cases from the Linux kernel dataset [23], excluding those used in the final evaluation [36]. We did not scale up this set due to the high cost of in-depth case analysis, as our goal was to perform detailed investigations to understand key challenges and inform component design. We conducted our study on the Linux kernel, a large and complex codebase that enables diverse and challenging cases. We investigated these cases while interacting with a LLM [2] to simulate the agent's behavior.

Through this process, we iteratively refined the tool design, domain knowledge, and context management. Initially, we employed `git_blame` and `git_show` as core tools following standard SZZ practices. These tools were implemented as thin wrappers with only essential parameters (e.g., commit hash and file path), which often produced excessive outputs and led to inefficient investigation. To address this, we introduced scoped parameters (e.g., line ranges, file filters, and search bounds) to constrain the search space. We further expanded the tool set to handle more complex scenarios by adding `git_log_s` for locating commits that introduce or remove specific identifiers, `git_grep` for repository-wide search, and `git_log_func` for function-level history tracing. We exclude `git bisect`, as it requires compiling and executing.

Beyond tool refinement, we distilled domain knowledge from these investigations and existing literature. This includes heuristics such as starting with blame as a default entry point while allowing the agent to switch to broader search tools (e.g., `git_log_s`) when necessary. We also encourage continued blame tracing when encountering refactoring [11] or meta-changes [10] (e.g., merges), preventing premature convergence to non-causal commits. These heuristics guide deeper exploration and enable the identification of bug-inducing commits that require non-trivial investigation. We further observed that tool outputs can be large, introducing unnecessary noise and increasing token consumption. This observation motivated the design of structured context compression to preserve essential evidence while reducing token usage.

3.2 Approach Overview

As shown in Figure 1, the overall workflow of AgentSZZ into three main phases: *Preprocessing*, *ReAct investigation loop*, and *Output*.

Preprocessing. Given a fix commit, AgentSZZ retrieves the commit message and diff of the BFC (1). Irrelevant information (e.g., `Reviewed-by` tags) is filtered to produce a concise content for the agent. To prevent information leakage, we remove any content that may reveal the BIC, including the `Fixes` tag in the Linux dataset [23] and BIC hashes in GitHub commit titles [30]. We then extract the fix commit date and use it as a hard temporal upper

Table 1: Task-specific tools provided to the AgentSZZ agent. Parameters marked with * are required. These tools are designed to cover the key operations involved in tracing bug-inducing commits.

Tool	Parameters
git_show	commit*, file_filter, stat_only, context_lines
git_blame	file_path*, commit, line_start, line_end
git_log_s	search_string*, path, after, before
git_log_func	function_name*, file_path*, after, before
git_grep	search_string*, commit, path

bound for all subsequent search operations (2). Finally, fix information, tool descriptions, domain knowledge, and search constraints are assembled into the initial context for the LLM agent (3).

ReAct Investigation Loop. The agent then enters a multi-turn investigation loop following the ReAct paradigm [48]. This iterative observe–reason–act cycle enables adaptive exploration of the repository, with observations and intermediate reasoning accumulated across turns. We set a maximum of 15 turns, following prior agent-based approaches [22, 37, 45]. Given the initial context, the agent first analyzes the available information and plans the next action based on domain knowledge (4). It then invokes one of the **task-specific tools** to interact with the repository and gather observations (5). The tool output is subsequently processed by the context compression module to reduce token consumption (6). Based on the compressed observations, the agent evaluates candidate commits according to domain knowledge and updates its belief about the BIC (7). This process repeats until the agent identifies a BIC candidate with sufficient evidence or reaches the turn limit, allowing it to iteratively refine its search and adapt its strategy when initial approaches prove unfruitful.

Output. Once the agent identifies a BIC candidate or reaches the turn limit, it produces a structured output containing the predicted BIC hash, confidence level, and supporting reasoning. AgentSZZ then parses this output (8) and normalizes the commit hash (9). The extracted hash is sanitized to remove non-hexadecimal characters and verified against the repository using `git rev-parse`. If the full hash cannot be resolved (e.g., due to hallucination), If no valid commit can be resolved, the prediction is discarded.

3.3 Task-Specific Tools

Table 1 summarizes the five tools provided to the agent. Instead of exposing raw git commands, we design a task-specific interface that constrains the agent's action space to operations relevant to bug-inducing commit identification.

Tool Abstraction. An unrestricted git interface exposes hundreds of subcommands and flag combinations, many of which are irrelevant to bug-inducing commit identification. We distill this space into five tools, each supporting a distinct operation in the investigation process (Section 3.1). `git_blame` traces line-level ownership to identify the commit that last modified a given line. `git_show` retrieves the commit message and diff for semantic classification (e.g., distinguishing refactoring from bug inducing). `git_log_s` (pickaxe search) and `git_log_func` (function-level history) support tracing code origins across file renames and structural changes. `git_grep` searches the codebase at a specific revision to locate definitions, call sites, and cross-file references. All five tools wrap standard Git commands (`git blame`, `git show`, `git log -S`, `git log -L`, and `git grep`), but are exposed as function-calling schemas via the API `tools` parameter, restricting the agent to invoking only these predefined operations with schema-conforming arguments.

AgentSZZ Prompt

You are an expert at identifying Bug-Introducing Commits (BICs) in the software repositories. Given a fix commit, find the commit that originally introduced the bug.

Tools

1. **git_show(commit, context_lines, stat_only, file_filter)** — View commit message and diff. Use file_filter for one file's changes in large commits.
2. **git_blame(file_path, commit, line_start, line_end)** — Show which commit last modified each line.
3. **git_log_s(search_string, path, after, before, use_regex, max_commits)** — Search commits by added/removed string (-S) or diff pattern (-G).
4. **git_log_func(function_name, file_path, after, before, max_commits)** — Track function body change history.
5. **git_grep(search_string, commit, path)** — Search codebase at a commit snapshot.

Workflow

Step 1: git_blame at fix~1
<Trace the changed lines to their last modifier>
Step 2: Evaluate Blame Results
<Classify: refactor → penetrate | introduction → BICambiguous → counterfactual test>
Step 3: Report
<Confirm causal link, output BIC with evidence>
<r>
BIC: <full_commit_hash>
confidence: <high|medium|low>
type: <introduction|omission|cross_file|refactor_penetration>
reasoning: <brief explanation>
</r> (The full workflow prompt is available in our replication package.)

Tool Rules

- git_log_s: include a path when possible; broaden only if empty. Use the fix commit date as the `before` upper bound — the BIC must predate the fix.
- git_show on large commits: use file_filter.
- Do not git_show the same commit more than twice.
- Timeout → retry with narrower parameters.
- file_filter returns "no changes" → file was at a different path in that commit. Try git_show WITHOUT file_filter to see the actual paths, or blame at commit~1.

Figure 2: Prompt for AgentSZZ, encoding domain knowledge for Bug-Inducing Commit (BIC) investigation.

Scoped Parameters. Each tool exposes a focused set of parameters that constrain its output scope, enabling more targeted queries. `git_blame` supports `line_start` and `line_end` to focus on modified lines, while `git_show` provides a `file_filter` to restrict diff output to specific files. The search tools (`git_log_s`, `git_log_func`) support temporal scoping via date parameters, allowing exploration within relevant history. `git_grep` accepts a `commit` parameter to search the codebase at a specific revision, and a `path` filter to narrow results to relevant directories. These parameters remain optional, allowing the agent to apply them based on context while preserving flexibility within a bounded search space.

3.4 Domain Knowledge

The agent's investigation is guided by a structured execution prompt that encodes domain knowledge distilled from our design process (Section 3.1) and existing literature. The prompt of AgentSZZ (Fig. 2) consists of three components: a task description, tool specifications (Section 3.3), and domain knowledge that defines both high-level investigation guidance and operational rules.

textbfInvestigation Workflow. The domain knowledge encodes a three-step investigation strategy as high-level guidance, while the agent retains full autonomy over tool selection, invocation order, and query parameters at each step. **Step 1 (Entry point selection).** The domain knowledge recommends starting with `git_blame` on

the modified or deleted lines at the parent of the bug-fixing commit (`fix~1`). For ghost commits (where the bug-fixing commit does not contain modified or deleted lines), our domain knowledge suggests blaming surrounding context lines. Moreover, we suggest that the agent use `git_log_s` or `git_log_func` to trace the history of relevant identifiers when blaming surrounding context does not work. For cross-file cases, we suggest that the agent may instead begin with broader search tools (e.g., `git_log_s`, `git_grep`). **Step 2 (Candidate analysis).** Candidate commits surfaced during exploration are classified into two categories: (i) *non-semantic changes*, including refactoring and meta-changes (e.g., merge, branch), which are bypassed via iterative re-blame at the candidate's parent commit to reach the true origin, following established SZZ practices such as MA-SZZ [11] and RA-SZZ [24]; and (ii) *semantic introductions*, which introduce or modify logic and are treated as likely bug-inducing commits. When multiple semantic candidates exist, the prompt provides a counterfactual heuristic for disambiguation: "would the bug exist if this commit had not been introduced?" **Step 3 (Causal confirmation).** Before reporting, the agent verifies a direct causal link between the selected candidate and the bug, ensuring it is causally responsible rather than merely correlated.

Rules. The prompt further encodes lightweight operational constraints that can be applied at the agent's discretion. These include prioritizing path-restricted queries before broadening scope, avoiding redundant inspection (e.g., limiting repeated `git_show` calls on the same commit), retrying with narrower parameters after timeouts, and accounting for file path changes across commits. It also encodes a key semantic distinction: commits that merely *expose* or *trigger* a pre-existing bug are not considered BICs; only commits that *introduce* faulty logic qualify.

3.5 Context Compression

To manage context consumption during multi-turn investigation, we design a multi-layer compression pipeline (Algorithm 1) that reduces tool output size while preserving high-signal evidence.

Search Bound Enforcement. Before tool execution, the system automatically caps the `before` parameter of temporal search tools to the fix commit date d_{fix}. Since a bug-inducing commit must predate its fix, this removes irrelevant results without requiring the agent to explicitly enforce this constraint.

Layer 1: Cache Deduplication. A call-level cache keyed by tool name and arguments returns previously computed results when queries are repeated, preventing duplicate outputs from accumulating across turns. **Timeout Recovery.** If a tool call times out, the system injects targeted hints (e.g., "*Retry with a smaller line range*") to guide the agent toward narrower queries, avoiding repeated expensive operations.

Layer 2: Formatted Outputs. Each tool applies internal formatting before returning results to improve readability and reduce noise. `git_blame` converts git's verbose porcelain format into a concise `L{n}: {hash} | {code}` representation with a commit legend. All commit-displaying tools filter out metadata trailers (e.g., `Signed-off-by`, `Reviewed-by`) that are irrelevant to bug-inducing commit identification. Additionally, each tool enforces a maximum output length (100–300 lines) with truncation notifications, enabling the agent to iteratively refine queries with narrower scopes.

Algorithm 1 Structured Context Compression

Require: Tool name t, arguments $args$, fix date d_{fix}, threshold τ
Ensure: Compressed output O'

// Search bound enforcement
if $t \in \{\text{log\_s}, \text{log\_func}\}$ **and** $args.\text{before} > d_{fix}$ **then**
  $args.\text{before} \leftarrow d_{fix}$
end if
// Layer 1: Cache deduplication
if $\textsc{Cache.Has}(t, args)$ **then**
  $O_{\text{raw}} \leftarrow \textsc{Cache.Get}(t, args)$
else
  $O_{\text{raw}} \leftarrow \textsc{ExecuteTool}(t, args)$
  if O_{raw} is timeout **then**
    Inject hint: "*Retry with narrower parameters*"
    return timeout message
  end if
  $\textsc{Cache.Store}(t, args, O_{\text{raw}})$
end if
// Layer 2: Formatted output
$O \leftarrow \textsc{Format}(t, O_{\text{raw}})$:
  Reformat raw output into concise representation
  Strip metadata trailers; truncate at max line limit
// Layer 3: Structured extraction
if $|O| \leq \tau$ **then return** O
end if
$O' \leftarrow \textsc{Extract}(t, O)$:
  show: Strip diff context lines (keep only `+`/`-`/`@@`); head-tail truncate at K_1
  blame: Deduplicate commits into summary header; head-tail truncate at K_2
  log: Keep first K_3 entries
  grep: Group by file; keep first K_4 matches
return O'

The truncation thresholds (100–300 lines) are set based on the typical output volume of each tool: `git_grep` produces many short matching lines and is capped at 100 to avoid flooding the context with repetitive results, while `git_log_func` includes inline diffs that require more space and is allowed up to 300 lines. When output exceeds the threshold, the tool appends a truncation notification, prompting the agent to retry with narrower parameters (e.g., adding a path filter or line range). We did not perform systematic tuning of these thresholds; they are designed to balance information completeness with context window efficiency.

Layer 3: Structured Extraction. When formatted outputs exceed a character threshold $\tau = 3000$, a deterministic, tool-specific extractor further compresses them. For `git_show`, diff context lines are removed, retaining only change markers (`+`/`-`/`@@`) and file headers with head-tail truncation at K_1 lines. For `git_blame`, repeated commits are deduplicated into a summary header, followed by head-tail truncation at K_2 lines. For `git_log` variants, only the first K_3 entries are retained, and for `git_grep`, matches are grouped by file before limiting to K_4 entries. These rules preserve structurally important information (e.g., commit hashes, line numbers, and changed code) while aggressively removing redundant context.

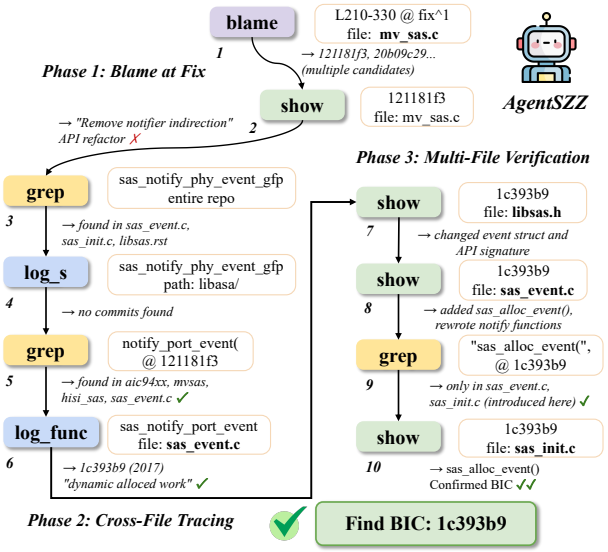

Figure 3: A successful case of AgentSZZ identifying a cross-file bug-inducing commit.

3.6 Successful Example

We illustrate AgentSZZ through a cross-file bug in the Linux kernel's SCSI subsystem.[1] Although the fix modifies `mvsas/mv_sas.c`, the root cause lies in the core library `libsas/sas_event.c`, where the event handling mechanism was rewritten four years earlier. Fig. 3 shows the 10-turn investigation trajectory.

Phase 1: Blame at fix (Turns 1–2). Following the blame-first strategy, the agent blames the changed lines in `mv_sas.c` at the parent commit and identifies commit `121181f3` as a frequent modifier. `git_show` reveals this commit to be an API refactoring ("Remove notifier indirection") that replaces indirect callbacks with direct calls—a mechanical change rather than the root cause.

Phase 2: Cross-file tracing (Turns 3–6). Recognizing that the bug lies beyond the driver file, the agent shifts to cross-file exploration. It uses `git_grep` to locate sas_notify_phy_event_gfp across the repository, identifying references in `sas_event.c`, `sas_init.c`, and related files. After an unsuccessful `git_log_s` search, the agent adapts by tracing the pre-refactoring function `notify_port_event`. Using `git_log_func`, it identifies commit `1c393b9` (2017) as the change that rewrote the event handling logic.

Phase 3: Multi-file verification (Turns 7–10). The agent verifies the candidate across multiple files. It confirms changes to the API and data structures in `libsas.h`, validates the shift to dynamic allocation in `sas_event.c`, and uses `git_grep` to confirm that `sas_alloc_event` was introduced in this commit. Finally, it inspects `sas_init.c` to complete the verification.

The agent correctly identifies `1c393b9` as the bug-inducing commit with high confidence. This case highlights three key strengths of AgentSZZ: (i) *domain knowledge* enables effective refactor penetration and strategy adaptation; (ii) *task-specific tools* support cross-file tracing beyond line-based SZZ; and (iii) *context compression* maintains tractable context across multi-turn investigation. This trajectory illustrates how AgentSZZ progressively shifts from local blame to broader repository exploration when local evidence becomes insufficient. Notably, the BIC resides in a different file and predates the fix by four years—conditions under which traditional SZZ approaches typically fail.

[1] https://github.com/torvalds/linux/commit/feb18e900f0048001ff375dca639eaa327ab3

4 Study Design

4.1 Research Questions

To evaluate AgentSZZ, we design the following research questions:

- **RQ1.** How effective is AgentSZZ at identifying BICs compared to baselines?
- **RQ2.** How efficient is AgentSZZ in terms of time and cost?
- **RQ3.** What is the contribution of each key component of AgentSZZ?
- **RQ4.** Can AgentSZZ identify BICs in challenging scenarios?

In RQ1, we evaluate the effectiveness of AgentSZZ by comparing it with a range of existing SZZ algorithms on the task of identifying bug-inducing commits across multiple datasets. In RQ2, we assess the efficiency of AgentSZZ in terms of runtime, token consumption, and monetary cost, and compare it against the state-of-the-art LLM-based approach LLM4SZZ. In RQ3, we conduct an ablation study to quantify the contribution of each key component of AgentSZZ (i.e., task-specific tools, domain knowledge, and context compression) to its overall performance and efficiency. In RQ4, we further analyze the effectiveness of AgentSZZ in challenging scenarios, with a focus on cross-file and ghost commit cases that are known to be difficult for traditional SZZ methods [23, 29].

4.2 Evaluation Dataset

To evaluate AgentSZZ, we use the same datasets as Tang et al. [36], including DS_LINUX, DS_GITHUB, and DS_APACHE. DS_LINUX contains 1,500 bug-fixing commits from the Linux kernel, sampled from the dataset of Lyu et al. [23]. DS_GITHUB contains 361 bug-fixing commits from 285 GitHub repositories covering C and Java projects [30]. During our evaluation (March 2026), we removed 6 unavailable repositories, leaving 355 bug-fixing commits from 279 repositories. DS_APACHE contains 241 bug-fixing commits from five Apache projects [42]. In total, we evaluate AgentSZZ on 2,095 bug-fixing commits with 2,271 bug-inducing commits.

4.3 Baselines

We select nine SZZ algorithms discussed in Section 2 as baselines to compare with AgentSZZ: B-SZZ [33], AG-SZZ [18], MA-SZZ [10], R-SZZ [12], L-SZZ [12], RA-SZZ [24], Neural-SZZ [35], TC-SZZ [23], and LLM4SZZ [36], following prior studies [23, 36]. Neural-SZZ and RA-SZZ are limited to Java projects, so we just compare in the datasets with Java projects. TG-SZZ [31] are excluded as they did not release their implementation.

To ensure a fair comparison, we run LLM4SZZ using the same LLM as AgentSZZ (GPT-5-mini [26]), denoted as LLM4SZZ (5-mini) in our evaluation. We also include mini-swe-agent [47] as a baseline, as it represents an agent-based software engineering

framework that supports multi-step reasoning and tool use, making it suitable for complex repository analysis tasks. Overall, we compare AgentSZZ against eleven baselines.

4.4 Evaluation Metrics

Following prior work on SZZ evaluation [36], we adopt precision, recall, and F1-score to measure bug-inducing commit identification accuracy. Let $GT(c_i)$ and $Pred(c_i)$ denote the ground-truth and predicted bug-inducing commit sets for bug-fixing commit c_i. We compute metrics over all commits as follows:

$$Precision = \frac{\sum_i |GT(c_i) \cap Pred(c_i)|}{\sum_i |Pred(c_i)|} \tag{1}$$

$$Recall = \frac{\sum_i |GT(c_i) \cap Pred(c_i)|}{\sum_i |GT(c_i)|} \tag{2}$$

$$F1 = 2 \cdot \frac{Precision \times Recall}{Precision + Recall} \tag{3}$$

4.5 Implementation

Using state-of-the-art code LLMs (e.g., Claude Opus 4.6 [2]) is impractical for large-scale evaluation due to their high cost. A single evaluation run on our three datasets would cost approximately $400, making such models unsuitable for our study. While prior work [36] adopts GPT-4o-mini [25], we instead use GPT-5-mini [26], which provides stronger support for tool use and reasoning while remaining cost-effective.

All experiments are conducted on a server equipped with an AMD EPYC 7763 64-core processor and 252 GB of RAM. We access LLM APIs via OpenRouter.[2] We do not set the temperature to 0, as GPT-5 series models do not support temperature control [3]. The agent is limited to a maximum of 15 interaction turns.

For the mini-SWE-agent (v2.2.5) baseline [47], we repurpose its bash-based repository interaction environment for BIC tracing instead of its default software repair workflow. The agent uses GPT-5-mini via OpenRouter and is run with a 300-second timeout per case. To ensure a fair comparison, it is provided with the same fix-side evidence as AgentSZZ: the repository path, the fix commit hash, and a filtered fix context derived from the fixing commit, with metadata trailers (e.g., `Fixes:`) removed. The agent is prompted to trace the origin of the bug using read-only git commands and to output the final BIC in a structured format.

5 Results

5.1 RQ1: Effectiveness of AgentSZZ

Table 2 presents the results across three datasets. Following prior work [36], we further split the GitHub dataset into C and Java subsets to analyze language-specific performance. This results in four datasets: DS_LINUX (Linux), DS_GITHUB-c (GitHub-C), DS_GITHUB-j (GitHub-J), and DS_APACHE (Apache). To ensure a fair comparison, we run AgentSZZ, LLM4SZZ, and LLM4SZZ (5-mini) three times and report the average results. Due to its high cost and low performance, mini-swe-agent is evaluated with a single run on each dataset.

[2]https://openrouter.ai

AgentSZZ achieves the highest recall and F1-score across all four datasets among eleven baselines. Compared to the prior state-of-the-art LLM4SZZ [36], it improves F1 by 27.2%, 22.9%, 25.2%, and 15.4% on Linux, GitHub-C, GitHub-J, and Apache, respectively. It also achieves strong precision (0.788, 0.830, 0.750, and 0.724), corresponding to relative improvements of 25.5%, 20.8%, 23.6%, and 18.7% over LLM4SZZ. Although AgentSZZ has slightly lower precision, the extremely low recall of mini-swe-agent leads to misleadingly high precision due to its limited number of correct predictions.

Using the same backbone model (LLM4SZZ with GPT-5-mini), AgentSZZ improves F1 by 26.1%, 21.1%, 19.3%, and 13.9%, indicating that the gains stem from the agent design and domain knowledge rather than the choice of LLM. In contrast, mini-swe-agent [47] performs poorly across all datasets (F1: 0.123, 0.123, 0.123, 0.011), largely due to unfocused exploration. While it retrieves many related commits using general-purpose tools (e.g., `git show`, `git log`, `git grep`), it rarely forms a stable tracing process toward the true BIC. Successful cases involve short tool chains and early convergence, whereas failures exhibit long exploratory sequences with repeated inspection but limited progress. Occasional command construction errors further degrade performance.

We compare our results with both LLM4SZZ through static analysis, as all run three times and the results show that AgentSZZ consistently outperforms both LLM4SZZ with significant and effective size.p < 0.001

AgentSZZ also substantially improves recall. Compared to B-SZZ, recall increases by 23.0%, 22.0%, 5.9%, and 3.9% on Linux, GitHub-C, GitHub-J, and Apache, respectively, while maintaining high precision. The smaller gain on Apache is due to its higher average number of bug-inducing commits per fix (1.46 vs. 1.04 for Linux), where AgentSZZ tends to identify a single bug-inducing commit, limiting recall in multi-bug-inducing commits cases.

AgentSZZ further demonstrates strong consistency and generalizability. Despite being designed based on Linux, it achieves substantial improvements on GitHub and Apache, and generalizes across both C and Java without dataset-specific tuning.

> **Answers to RQ1**: AgentSZZ outperforms state-of-the-art methods in precision, recall, and F1-score across all datasets, achieving F1 improvements of 14.5%-27.2%. It consistently outperforms LLM4SZZ under the same backbone, while mini-swe-agent performs poorly due to unfocused exploration. AgentSZZ also generalizes well across datasets and programming languages.

5.2 RQ2. Efficiency of AgentSZZ

Table 3 compares the efficiency of four LLM-based approaches in terms of runtime, token consumption, interaction turns, and monetary cost. All API calls are made via OpenRouter to ensure a fair comparison, and all values are averaged over three datasets.

AgentSZZ achieves a strong balance between efficiency and effectiveness. Compared to LLM4SZZ, it incurs a modest increase in runtime (39.1s vs. 26.2s) and token usage (34,657 vs. 17,690), but requires substantially fewer interaction turns (5.7 vs. 11.2), resulting in a much higher per-turn information density (6,080 vs.

Table 2: Performance comparison of methods for identifying bug-inducing commits on C and Java datasets. RA-SZZ and Neural-SZZ are only applicable to Java, so their results are marked as "–" for C datasets. Ablation results are shown in the bottom rows; For the main methods, the best value in each column is bold and the second-best is underlined.

Method	DS_LINUX			DS_GITHUB-c			DS_GITHUB-j			DS_APACHE		
	Prec.	Rec.	F1	Prec.	Rec.	F1	Prec.	Rec.	F1	Prec.	Rec.	F1
B-SZZ	0.452	0.578	0.507	0.361	0.656	0.466	0.285	0.680	0.401	0.251	0.435	0.318
AG-SZZ	0.448	0.553	0.495	0.410	0.592	0.484	0.421	0.533	0.470	0.328	0.310	0.318
MA-SZZ	0.421	0.538	0.472	0.335	0.624	0.436	0.239	0.560	0.335	0.307	0.345	0.329
R-SZZ	0.583	0.448	0.507	0.671	0.582	0.620	0.538	0.467	0.500	0.497	0.288	0.364
L-SZZ	0.560	0.430	0.486	0.486	0.422	0.452	0.492	0.427	0.457	0.366	0.211	0.267
RA-SZZ	–	–	–	–	–	–	0.337	0.440	0.382	0.264	0.325	0.293
Neural-SZZ	–	–	–	–	–	–	0.556	0.486	0.520	0.563	0.364	0.442
LLM4SZZ	0.628	0.552	0.588	0.687	0.641	0.663	0.607	0.569	0.587	0.610	0.398	0.482
LLM4SZZ (5-mini)	0.580	0.607	0.593	0.691	0.655	0.673	0.696	0.600	0.616	0.623	0.401	0.488
mini-swe-agent	0.865	0.010	0.149	0.870	0.139	0.240	1.000	0.160	0.276	0.633	0.054	0.099
AgentSZZ	**0.788**	**0.711**	**0.748**	**0.830**	**0.800**	**0.815**	**0.750**	**0.720**	**0.735**	**0.724**	**0.452**	**0.556**
w/o Tools	0.212	0.125	0.157	0.009	0.007	0.008	0.029	0.027	0.028	0.000	0.000	0.000
w/o Domain	0.798	0.641	0.711	0.889	0.775	0.828	0.754	0.653	0.700	0.739	0.393	0.513
w/o Compression	0.792	0.746	0.768	0.812	0.789	0.801	0.841	0.773	0.806	0.706	0.469	0.564

Table 3: Cost analysis of AgentSZZ in terms of tokens and time consumption.

Method	Time (s)	Tokens	Turns	Cost ($)
LLM4SZZ	26.2	13,033	11.2	0.003
LLM4SZZ (5-mini)	125.2	18,230	12.6	0.015
mini-swe-agent	183.3	330,327	18.6	0.035
AgentSZZ	39.1	34,657	5.7	0.009
w/o Compression	29.8	52,237	4.7	0.014

1,580 tokens/turn). This indicates that AgentSZZ extracts significantly more information per step through structured tool use, and converges with fewer, more targeted interactions.

When LLM4SZZ is powered by GPT-5-mini, it becomes substantially slower (160s), due to higher latency per API call, while its token usage and interaction patterns remain largely unchanged. This suggests that stronger model reasoning alone does not improve efficiency in static pipelines. In contrast, AgentSZZ achieves both faster execution and higher accuracy by leveraging structured tool interaction rather than relying solely on internal reasoning. mini-swe-agent is significantly less efficient across all dimensions, consuming 4.7× more time, 9.5× more tokens, and 3.9× higher cost than AgentSZZ. Its higher turn count and excessive token usage reflect prolonged, unfocused exploration, a known limitation of open-ended bash-based agents. By contrast, AgentSZZ constrains the action space to curated git-based tools, enabling more directed and efficient investigation.

AgentSZZ achieves a strong balance between efficiency and performance. Compared to LLM4SZZ, it incurs only a modest increase in runtime (39.1s vs. 26.2s) and token usage, while using fewer interaction turns (5.7 vs. 11.2), indicating more efficient reasoning and faster convergence. Despite using more tokens than LLM4SZZ, the reduced number of turns suggests that AgentSZZ extracts more useful information per step through targeted tool usage. In contrast, mini-swe-agent is significantly less efficient, with much higher runtime (183.3s), token consumption (330k), and cost, due to prolonged and unfocused exploration. AgentSZZ reduces runtime by over 4× and token usage by nearly 10× compared to mini-swe-agent, while achieving substantially better accuracy.

> **Answers to RQ2**: AgentSZZ achieves a strong trade-off between efficiency and effectiveness. It requires only 5.7 turns on average, with a per-turn information density 3.8× higher than LLM4SZZ. Compared to mini-swe-agent, it reduces runtime, token usage, and cost by 4.7×, 9.5×, and 3.9×, respectively.

5.3 RQ3. Key Components of AgentSZZ

As shown in Table 2, the last three rows present the results of our ablation study, where we evaluate the contribution of each key component of AgentSZZ by removing it individually. We consider three variants: (1) *w/o Tools*, which removes all tool access and forces the agent to rely solely on fix commit information without interactive investigation; (2) *w/o Domain*, which replaces the domain knowledge prompt with minimal task instructions; and (3) *w/o Compression*, which disables all context compression module.

Structured tools are indispensable. Removing task-specific tools causes performance to collapse across all datasets, with F1 dropping from 0.748 to 0.157 on Linux, from 0.815 to 0.008 on GitHub-C, and to 0.000 on Apache. Without structured tool interfaces, the agent must compose commands, interpret raw outputs, and manage context within a limited turn budget—tasks that even

strong LLMs struggle with. This confirms that structured tools are the most critical component of AgentSZZ.

Domain knowledge consistently improves recall. Removing domain knowledge leads to consistent recall drops across all datasets, with the largest decrease observed on Apache (−13.0%), where commits are larger and more complex. On GitHub-C, removing domain knowledge slightly increases precision (0.830→0.889) but reduces recall, resulting in a marginal F1 increase (0.815→0.828), while all other cases show an F1 decrease. Overall, domain knowledge promotes broader exploration—occasionally introducing false positives but consistently improving recall—while generalizing across both C and Java.

Context compression improves efficiency with minimal impact on effectiveness. Disabling compression has limited impact on effectiveness, with F1 changes within ±2% on the three larger datasets. In contrast, its impact on efficiency is substantial: token consumption decreases by 33.6% (52,237→34,657) and cost by 35.7% ($0.014→$0.009). This highlights the importance of compression for practical deployment. We focus on the three larger datasets due to higher variance in GitHub-J.

Figure 4 shows the distribution of tool usage across investigation turns. Most cases are resolved within 4–6 turns, with `git_show` and `git_blame` accounting for approximately 40% and 18% of tool calls, respectively. Early turns are dominated by `git_blame`, while `git_show` peaks during candidate inspection. In later turns, the agent increasingly relies on broader search tools (`git_log_s` and `git_grep`), reflecting a transition from local blame to global search when initial evidence is insufficient. `git_log_func` is used sparingly as a fallback for function-level tracing. Exploratory tools (`git_grep`, `git_log_s`) appear more frequently in harder scenarios (e.g., omission, refactor-penetration, cross-file cases), where they serve as escalation mechanisms for cross-file and semantic tracing. `git_log_func` plays a minor role as a specialized fallback.

> **Answers to RQ3**: Designed Tools are indispensable, with F1 dropping to near zero without them. Domain knowledge improves recall, while context compression reduces token consumption by over 30% with minimal impact on effectiveness.

5.4 RQ4. Challenging Scenarios

To evaluate AgentSZZ in challenging scenarios, we focus on bug-fixing commits categorized as cross-file and ghost cases (Table 4). Following Lyu et al. [23], a case is defined as cross-file if none of its bug-inducing commits appear in the history of files modified by the fix (via `git log`), and as a ghost case if the bug-fixing commit contains no deleted or modified lines.

The results show that our approach significantly outperforms LLM4SZZ in both cross-file and ghost commit scenarios. For cross-file cases, AgentSZZ achieves recall of 18.60%, 24.24%, and 42.72% on Apache, GitHub, and Linux, respectively, corresponding to improvements of 100.0%, 300.0%, and 160.1%.

LLM4SZZ can handle a small number of cross-file cases, but its success mainly stems from indirect coverage rather than explicit cross-file reasoning. Its pipeline remains centered on a single fix-side file, where buggy statements are extracted and traced within the same file history. As a result, it only succeeds when sufficient local signals remain (e.g., due to file renaming or residual context in the patch). In contrast, AgentSZZ is not restricted to a single file history. Starting from a `git_blame` → `git_show` backbone, it can escalate to `git_grep` and `git_log_s` to bridge symbol-level, statement-level, and cross-file relationships, enabling reliable transitions from the fix location to the true origin.

A similar pattern explains the gap on ghost commits. LLM4SZZ relies on explicit line-level signals from the fixing patch to recover candidate commits. When no lines are deleted or modified, this signal becomes unavailable, significantly weakening the pipeline. By contrast, AgentSZZ leverages broader context, semantic reasoning, and tool-assisted exploration to identify the bug-inducing commit even without direct line-level evidence, making it substantially more robust in ghost scenarios.

> **Answers to RQ4**: Our approach significantly outperforms LLM4SZZ in challenging scenarios, improving recall by up to 300% on cross-file cases and 60% on ghost commits.

6 Discussion

6.1 Failure Analysis

We conduct a qualitative error analysis on a sample of 50 incorrect cases from Apache, GitHub, and Linux. The dominant failure mode is misattribution among plausible related commits. Specifically, 24/50 errors are local semantic misattributions, where the agent identifies a nearby plausible commit but fails to trace back to the earliest bug-inducing commit, and 11/50 stop at later related commits (e.g., propagation, follow-up, or refactoring steps).

Cross-file localization misses account for 5/50 cases, indicating limited pivoting beyond the fix-side file history in some instances. Empty-output failures are less frequent (8/50) but involve substantially longer trajectories, suggesting continued exploration without successful convergence. Notably, 40/50 errors are assigned high confidence, indicating that the remaining challenge lies in precise historical attribution rather than repository exploration.

These patterns are illustrated by representative cases. In an Apache case,[3] the agent remained anchored to the fix-side file history and selected a plausible blame-linked commit instead of the true cross-file origin. In a GitHub case[4]https://github.com/sebastiaanschool/sebastiaanschool-Android/commit/2454879bfb the agent chose a highly relevant but later related commit rather than the earliest introducing one. In a Linux ghost commit case,[5] weak fix-side signals led to a high-confidence but non-earliest attribution.

6.2 Threats to Validity

Internal Validity. LLMs may produce non-deterministic outputs even with temperature set to 0, potentially affecting reproducibility. We mitigate this by running each experiment three times and reporting average results. The domain knowledge in our prompt was derived from 50 Linux kernel cases. Although results show strong cross-dataset and cross-language generalizability, the prompt may encode Linux-specific patterns that are less relevant to other projects.

[3]https://github.com/apache/accumulo/commit/a2c2d38aa

[4],

[5]https://github.com/torvalds/linux/commit/7ecb37f62fe5

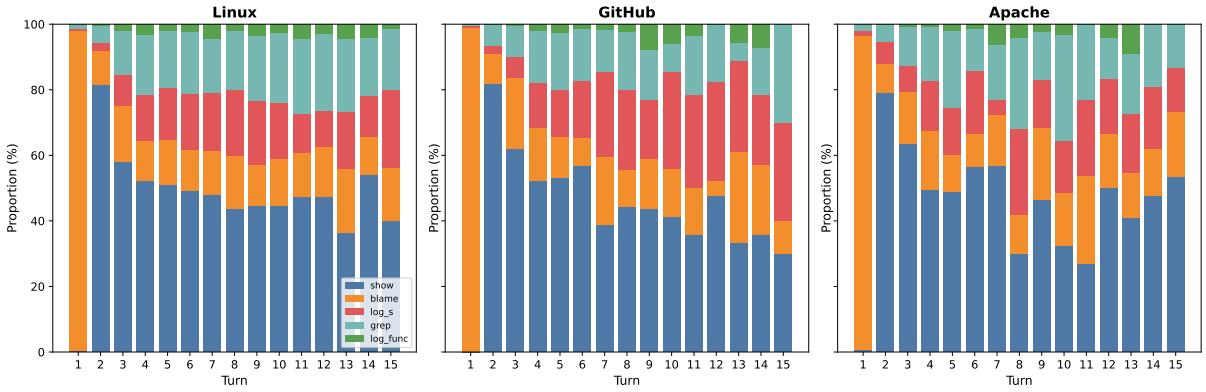

Figure 4: Distribution of tool usage across investigation steps in AgentSZZ. The values are normalized proportions per turn.

Table 4: Distribution, recall, and relative improvement of AgentSZZ over LLM4SZZ on cross-file and ghost commit cases.

Dataset	Total	Cross-file	Ghost	Cross-file Recall			Ghost Recall		
				AgentSZZ	LLM4SZZ	Improv.	AgentSZZ	LLM4SZZ	Improv.
Apache	243	34 (14.0%)	10 (4.1%)	18.60	9.30	+100.0%	53.33	40.00	+33.3%
GitHub	355	33 (9.3%)	37 (10.2%)	24.24	6.06	+300.0%	70.27	48.65	+44.4%
Linux	1500	209 (13.9%)	263 (17.5%)	42.72	16.43	+160.1%	64.18	39.93	+60.7%
Overall	2104	276 (13.1%)	310 (14.7%)	37.02	14.19	+160.9%	64.38	40.94	+57.2%

Future work could explore project-adaptive prompt generation to further improve robustness. The design process involved two authors interacting with an LLM to simulate agent behavior, introducing potential subjectivity in tool and prompt design. To mitigate this, the 50 design cases were excluded from all evaluation datasets, and all design choices were validated through ablation studies (RQ3). Prior work has shown that potential data leakage in similar settings does not materially affect evaluation outcomes. Therefore, we do not conduct additional experiments on this aspect, as it is orthogonal to the focus of this study.

In addition, the performance of AgentSZZ depends on the quality of tool outputs and repository metadata (e.g., commit history and file structure). Incomplete or inconsistent repository information may affect the accuracy of tracing results. AgentSZZ attempts prefix matching at progressively shorter lengths (12, 10, 8, 7 characters) to recover from minor errors.

External Validity. Our evaluation spans three datasets, two programming languages (C and Java), and over 285 repositories, covering a broader scope than most prior SZZ studies. However, generalizability to other languages (e.g., Python, JavaScript) and project types (e.g., web or mobile applications) remains to be validated. We use developer-annotated ground truth, the highest-quality labeling available for SZZ evaluation. However, annotations may still contain errors or ambiguities, especially for complex bugs involving multiple commits. Additionally, DS_GITHUB excludes six repositories due to unavailability during our evaluation period, which may slightly affect comparability with prior work. Finally, our evaluation focuses on open-source projects; the applicability of AgentSZZ to proprietary or industrial codebases with different development practices remains an open question.

7 Conclusion and Future Work

In this paper, we propose AgentSZZ, an agent-based framework that leverages LLM-driven agents to identify bug-inducing commits through interactive repository investigation. Unlike static *git blame* pipelines, AgentSZZ combines task-specific tools, domain knowledge, and a ReAct-style loop to enable adaptive, evidence-driven exploration of the repository. A structured compression module further reduces token consumption by over 30% with negligible impact, improving efficiency without sacrificing effectiveness.

Extensive experiments on three developer-annotated datasets show that AgentSZZ outperforms all eleven baselines, achieving F1 improvements of 15.4%–27.2% over LLM4SZZ. The gains are most pronounced in challenging scenarios: recall improves by up to 300% on cross-file cases and 60% on ghost commits, where static tracing methods often fail. Ablation studies show that task-specific tools are indispensable, domain knowledge significantly improves recall, and the overall architecture generalizes well across multiple LLM backbones. These results suggest that structured tool interaction, rather than purely model-based reasoning, is key to effective and efficient bug-inducing commit identification.

Several directions remain for future work. The dominant failure mode is misattribution among plausible commits rather than localization errors, suggesting the need for finer-grained temporal reasoning or stronger causal verification. For example, incorporating execution-based validation could help distinguish commits that introduce faults from those that merely expose them. AgentSZZ

currently tends to identify a single BIC per fix, limiting recall in multi-BIC cases (e.g., DS_APACHE). Future work could extend the framework to support multiple BICs by modeling dependencies across commits. Future work could also explore automated domain knowledge extraction and integrate execution or static analysis to better verify causal relationships.

References

[1] 2026. ACM SIGSOFT Impact Paper Award. https://www2.sigsoft.org/awards/impactpaper/.

[2] Anthropic. 2026. Claude Opus 4.6 System Card. https://www.anthropic.com/system-cards

[3] ashwinthandu03. 2025. GPT-5 models – Temperature. OpenAI Developer Community. https://community.openai.com/t/gpt-5-models-temperature/1337957 Accessed: 2026-03-25.

[4] Lingfeng Bao, Xin Xia, Ahmed E. Hassan, and Xiaohu Yang. 2022. V-SZZ: automatic identification of version ranges affected by CVE vulnerabilities. In *Proceedings of the 44th International Conference on Software Engineering* (Pittsburgh, Pennsylvania) *(ICSE '22)*. Association for Computing Machinery, New York, NY, USA, 2352–2364. doi:10.1145/3510003.3510113

[5] Gabriele Bavota and Barbara Russo. 2015. Four eyes are better than two: On the impact of code reviews on software quality. In *2015 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 81–90. doi:10.1109/ICSM.2015.7332454

[6] Boyuan Chen and Zhen Ming Jiang. 2019. Extracting and studying the Logging-Code-Issue-Introducing changes in Java-based large-scale open source software systems. *Empirical Software Engineering* 24, 4 (2019), 2285–2322.

[7] Junkai Chen, Huihui Huang, Yunbo Lyu, Junwen An, Jieke Shi, Chengran Yang, Ting Zhang, Haoye Tian, Yikun Li, Zhenhao Li, et al. 2025. SecureAgentBench: Benchmarking Secure Code Generation under Realistic Vulnerability Scenarios. *arXiv preprint arXiv:2509.22097* (2025).

[8] Xingchu Chen, Chengwei Liu, Jialun Cao, Yang Xiao, Xinyue Cai, Yeting Li, Jingyi Shi, Tianqi Sun, et al. 2025. Vulnerability-Affected Versions Identification: How Far Are We? *arXiv preprint arXiv:2509.03876* (2025).

[9] Cognition AI. 2024. Devin, AI software engineer. https://www.cognition.ai/introducing-devin.

[10] Daniel Alencar da Costa, Shane McIntosh, Weiyi Shang, Uirá Kulesza, Roberta Coelho, and Ahmed E. Hassan. 2017. A Framework for Evaluating the Results of the SZZ Approach for Identifying Bug-Introducing Changes. *IEEE Transactions on Software Engineering* 43, 7 (2017), 641–657. doi:10.1109/TSE.2016.2616306

[11] Daniel Alencar da Costa, Shane McIntosh, Weiyi Shang, Uirá Kulesza, Roberta Coelho, and Ahmed E. Hassan. 2017. A Framework for Evaluating the Results of the SZZ Approach for Identifying Bug-Introducing Changes. *IEEE Transactions on Software Engineering* 43, 7 (2017), 641–657. doi:10.6084/m9.figshare.31869097

[12] Steven Davies, Marc Roper, and Murray Wood. 2014. Comparing text-based and dependence-based approaches for determining the origins of bugs. *Journal of Software: Evolution and Process* 26, 1 (2014), 107–139.

[13] Yuanrui Fan, Xin Xia, Daniel Alencar Da Costa, David Lo, Ahmed E Hassan, and Shanping Li. 2019. The impact of mislabeled changes by szz on just-in-time defect prediction. *IEEE transactions on software engineering* 47, 8 (2019), 1559–1586.

[14] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155* (2020).

[15] Hideaki Hata, Osamu Mizuno, and Tohru Kikuno. 2012. Bug prediction based on fine-grained module histories. In *2012 34th International Conference on Software Engineering (ICSE)*. 200–210. doi:10.1109/ICSE.2012.6227193

[16] Junda He, Christoph Treude, and David Lo. 2025. LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision and the Road Ahead. *ACM Transactions on Software Engineering and Methodology* (2025).

[17] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2023. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770* (2023).

[18] Sunghun Kim, Thomas Zimmermann, Kai Pan, and E. James Jr. Whitehead. 2006. Automatic Identification of Bug-Introducing Changes . In *Proceedings. 21st IEEE International Conference on Automated Software Engineering*. IEEE Computer Society, Los Alamitos, CA, USA, 81–90. doi:10.1109/ASE.2006.23

[19] Thomas D. LaToza, Gina Venolia, and Robert DeLine. 2006. Maintaining mental models: a study of developer work habits. In *Proceedings of the 28th International Conference on Software Engineering* (Shanghai, China) *(ICSE '06)*. Association for Computing Machinery, New York, NY, USA, 492–501. doi:10.1145/1134285.1134355

[20] Joseph Lawrance, Christopher Bogart, Margaret Burnett, Rachel Bellamy, Kyle Rector, and Scott D. Fleming. 2013. How Programmers Debug, Revisited: An Information Foraging Theory Perspective. *IEEE Transactions on Software Engineering* 39, 2 (2013), 197–215. doi:10.1109/TSE.2010.111

[21] Junwei Liu, Kaixin Wang, Yixuan Chen, Xin Peng, Zhenpeng Chen, Lingming Zhang, and Yiling Lou. 2026. Large Language Model-Based Agents for Software Engineering: A Survey. *ACM Trans. Softw. Eng. Methodol.* (March 2026). doi:10.1145/3796507 Just Accepted.

[22] Shuyang Liu, Yang Chen, Rahul Krishna, Saurabh Sinha, Jatin Ganhotra, and Reyhan Jabbarvand. 2025. Process-Centric Analysis of Agentic Software Systems. *arXiv preprint arXiv:2512.02393* (2025).

[23] Yunbo Lyu, Hong Jin Kang, Ratnadira Widyasari, Julia Lawall, and David Lo. 2024. Evaluating SZZ Implementations: An Empirical Study on the Linux Kernel. *IEEE Trans. Softw. Eng.* 50, 9 (Sept. 2024), 2219–2239. doi:10.1109/TSE.2024.3406718

[24] Edmilson Campos Neto, Daniel Alencar da Costa, and Uirá Kulesza. 2018. The impact of refactoring changes on the SZZ algorithm: An empirical study. In *2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 380–390. doi:10.1109/SANER.2018.8330225

[25] OpenAI. 2024. GPT-4o-mini. https://platform.openai.com/docs/models#gpt-4o-mini.

[26] OpenAI. 2025. GPT-5 mini. https://developers.openai.com/api/docs/models/gpt-5-mini.

[27] Chris Parnin and Alessandro Orso. 2011. Are automated debugging techniques actually helping programmers?. In *Proceedings of the 2011 International Symposium on Software Testing and Analysis* (Toronto, Ontario, Canada) *(ISSTA '11)*. Association for Computing Machinery, New York, NY, USA, 199–209. doi:10.1145/2001420.2001445

[28] Luca Pascarella, Fabio Palomba, and Alberto Bacchelli. 2019. Fine-grained just-in-time defect prediction. *Journal of Systems and Software* 150 (2019), 22–36. doi:10.1016/j.jss.2018.12.001

[29] Christophe Rezk, Yasutaka Kamei, and Shane McIntosh. 2022. The Ghost Commit Problem When Identifying Fix-Inducing Changes: An Empirical Study of Apache Projects. *IEEE Transactions on Software Engineering* 48, 9 (2022), 3297–3309. doi:10.1109/TSE.2021.3087419

[30] Giovanni Rosa, Luca Pascarella, Simone Scalabrino, Rosalia Tufano, Gabriele Bavota, Michele Lanza, and Rocco Oliveto. 2021. Evaluating SZZ Implementations Through a Developer-Informed Oracle. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. 436–447. doi:10.1109/ICSE43902.2021.00049

[31] Yu Shi, Hao Li, Bram Adams, and Ahmed E Hassan. 2026. Beyond Blame: Rethinking SZZ with Knowledge Graph Search. *arXiv preprint arXiv:2602.02934* (2026).

[32] Danilo Silva and Marco Tulio Valente. 2017. RefDiff: Detecting Refactorings in Version Histories. In *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*. 269–279. doi:10.1109/MSR.2017.14

[33] Jacek Śliwerski, Thomas Zimmermann, and Andreas Zeller. 2005. When do changes induce fixes? *SIGSOFT Softw. Eng. Notes* 30, 4 (May 2005), 1–5. doi:10.1145/1082983.1083147

[34] Ming Tan, Lin Tan, Sashank Dara, and Caleb Mayeux. 2015. Online Defect Prediction for Imbalanced Data. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, Vol. 2. 99–108. doi:10.1109/ICSE.2015.139

[35] Lingxiao Tang, Lingfeng Bao, Xin Xia, and Zhongdong Huang. 2023. Neural SZZ Algorithm. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 1024–1035. doi:10.1109/ASE56229.2023.00037

[36] Lingxiao Tang, Jiakun Liu, Zhongxin Liu, Xiaohu Yang, and Lingfeng Bao. 2025. LLM4SZZ: Enhancing SZZ Algorithm with Context-Enhanced Assessment on Large Language Models. *Proc. ACM Softw. Eng.* 2, ISSTA, Article ISSTA016 (June 2025), 23 pages. doi:10.1145/3728885

[37] Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, SH Cai, Yuan Cao, Y Charles, HS Che, Cheng Chen, Guanduo Chen, et al. 2026. Kimi K2. 5: Visual Agentic Intelligence. *arXiv preprint arXiv:2602.02276* (2026).

[38] Nikolaos Tsantalis, Matin Mansouri, Laleh M. Eshkevari, Davood Mazinanian, and Danny Dig. 2018. Accurate and efficient refactoring detection in commit history. In *Proceedings of the 40th International Conference on Software Engineering* (Gothenburg, Sweden) *(ICSE '18)*. Association for Computing Machinery, New York, NY, USA, 483–494. doi:10.1145/3180155.3180206

[39] Michele Tufano, Gabriele Bavota, Denys Poshyvanyk, Massimiliano Di Penta, Rocco Oliveto, and Andrea De Lucia. 2017. An empirical study on developer-related factors characterizing fix-inducing commits. *Journal of Software: Evolution and Process* 29, 1 (2017), e1797.

[40] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. 2024. Executable code actions elicit better llm agents. In *Forty-first International Conference on Machine Learning*.

[41] Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. 2024. Openhands: An open platform for ai software developers as generalist agents. *arXiv preprint arXiv:2407.16741* (2024).

[42] Ming Wen, Rongxin Wu, Yepang Liu, Yongqiang Tian, Xuan Xie, Shing-Chi Cheung, and Zhendong Su. 2019. Exploring and exploiting the correlations between bug-inducing and bug-fixing commits. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium*

on the Foundations of Software Engineering (Tallinn, Estonia) *(ESEC/FSE 2019)*. Association for Computing Machinery, New York, NY, USA, 326–337. doi:10.1145/3338906.3338962

[43] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. 2025. The rise and potential of large language model based agents: A survey. *Science China Information Sciences* 68, 2 (2025), 121101.

[44] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2025. Demystifying LLM-Based Software Engineering Agents. *Proc. ACM Softw. Eng.* 2, FSE, Article FSE037 (June 2025), 24 pages. doi:10.1145/3715754

[45] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh J Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. 2024. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *Advances in Neural Information Processing Systems* 37 (2024), 52040–52094.

[46] Meng Yan, Xin Xia, Yuanrui Fan, Ahmed E. Hassan, David Lo, and Shanping Li. 2022. Just-In-Time Defect Identification and Localization: A Two-Phase Framework. *IEEE Transactions on Software Engineering* 48, 1 (2022), 82–101. doi:10.1109/TSE.2020.2978819

[47] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., 50528–50652. https://proceedings.neurips.cc/paper_files/paper/2024/file/5a7c947568c1b1328ccc5230172e1e7c-Paper-Conference.pdf

[48] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*.

[49] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. 2023. RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 2471–2484. doi:10.18653/v1/2023.emnlp-main.151

[50] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. AutoCodeRover: Autonomous Program Improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Vienna, Austria) *(ISSTA 2024)*. Association for Computing Machinery, New York, NY, USA, 1592–1604. doi:10.1145/3650212.3680384